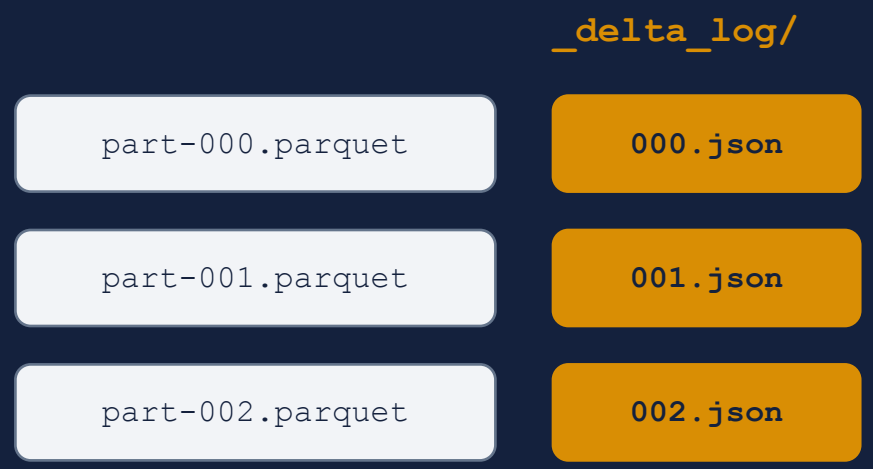
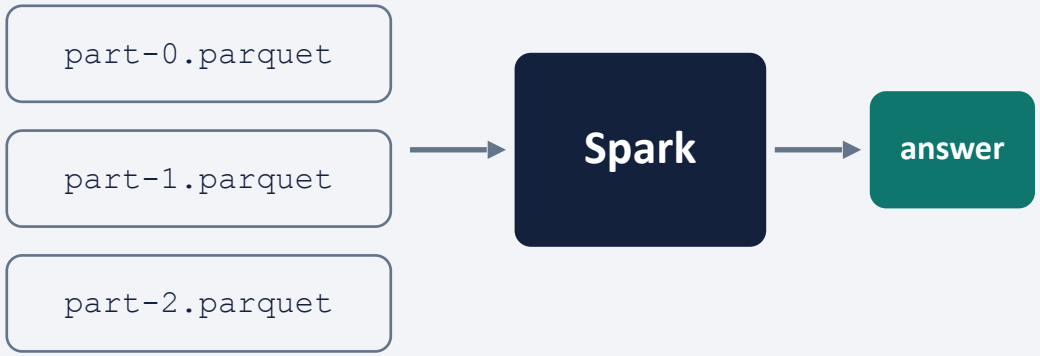
Delta Lake

From a folder of files to a table you can trust



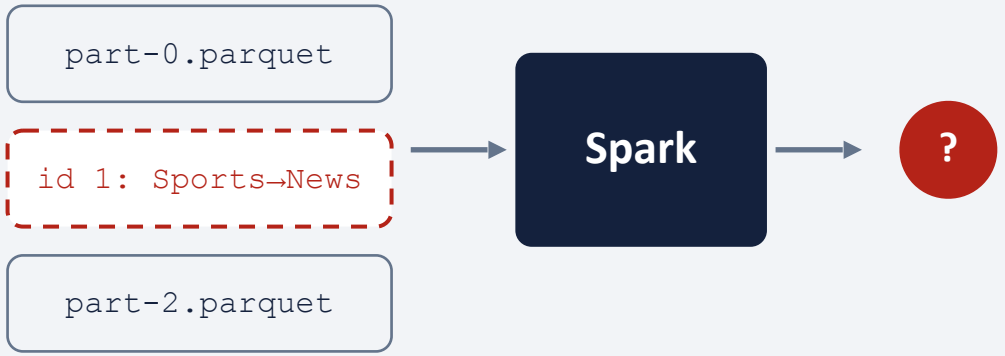
Last time: the engine. Today: the data changes

LECTURE 1



Read files, plan the query, compute an answer.

TODAY



Interests change. A wrong age arrives. A user leaves.

Spark is the engine. Today is about the table it reads and writes.

One table for the whole hour

id	gender	age	interest
1	Male	25	Sports
2	Female	30	News
3	Male	35	Social
4	Female	28	Fashion

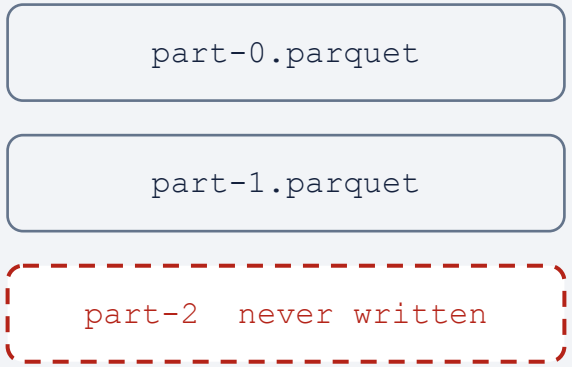
interest is one of Sports, News, Social, Fashion

What will happen to it

1. **User 1 changes interest**
   UPDATE and DELETE on files you can't edit
2. **A partner sends age 999**
   Versions, ACID, time travel
3. **The table grows to millions of rows**
   Data skipping
4. **A batch of new and changed users**
   MERGE
5. **A dashboard has to keep up**
   Change Data Feed

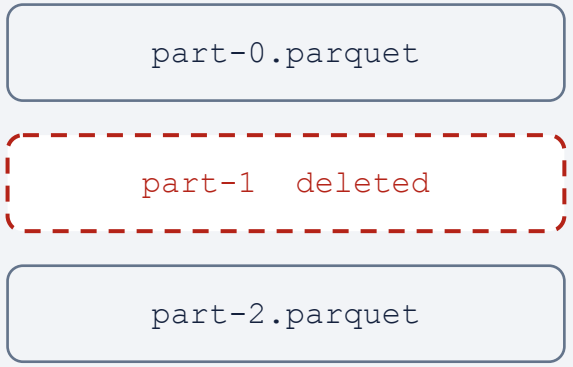
A folder of Parquet files is not a table

1 Half a write



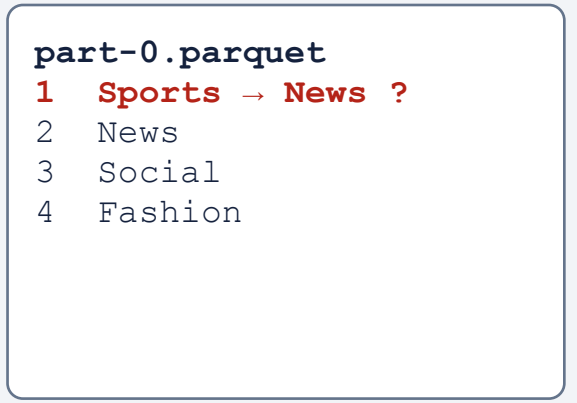
The job dies after 2 of 3 files. Readers see two thirds of the data, and nobody tells them.

2 A file disappears



Someone cleans up the folder. Readers silently get fewer rows. No error.

3 Change one row



Parquet can't be edited, so the file is rewritten. Mid-rewrite, readers may see old, new, both or neither.

The only definition of "the table" is whatever files are in the folder right now

What we want back

Databases have given us these for decades.

Data lakes gave them up for cheap storage that scales.

Delta Lake gets them back, on the same files.

- ✓ Change a single row — Module 1
- ✓ Readers never see half a change — Module 1
- ✓ Two writers can't silently clobber each other — Module 2
- ✓ Get back to yesterday's version — Module 2
- ✓ Don't read files we don't need — Module 3

Delta = Parquet + a log

profiles/

part-000.parquet

part-001.parquet

your data, ordinary Parquet

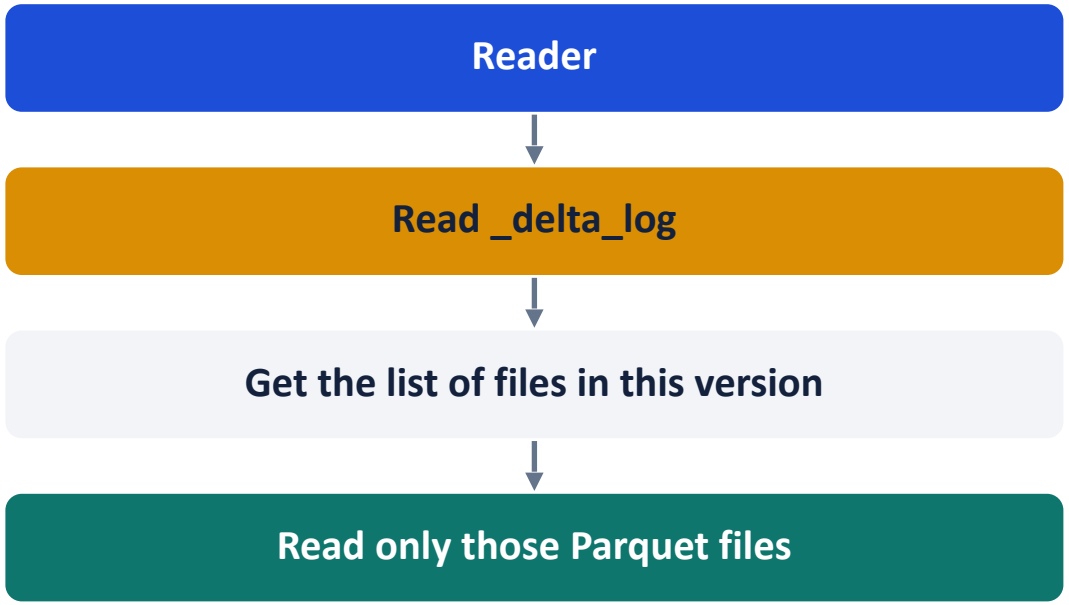
_delta_log/

000…000.json

000…001.json

one JSON file per version

How a reader reads it



A Delta table is Parquet files plus a log that says which files are the table right now

Switch to notebook 01

What the log just gave us

```
001.json
remove part-A
add    part-B
```

Update = remove + add, in one commit

A reader never sees half a change

```
stray.parquet

Parquet: 12 rows
Delta:    4 rows
```

A stray file was ignored

A crashed write can't pollute the table

v0 v1 v2

Old files kept, versions numbered

We can go back in time · Module 2

```
numRecords 4
min id 1
max id 4
```

Min and max stats in every add

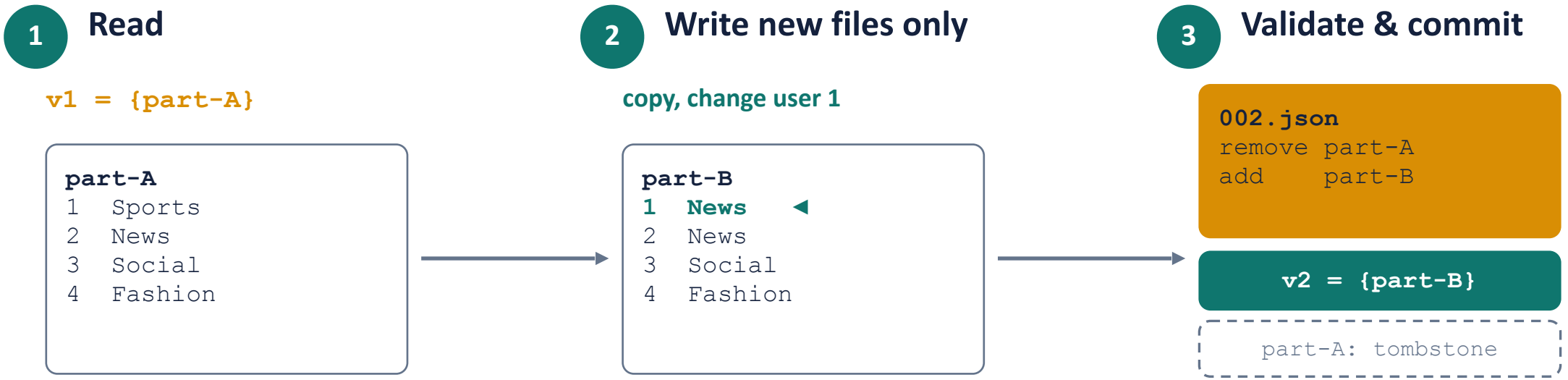
We can skip files · Module 3

Versions and rules

Many versions, two writers at once, and going back in time



MVCC: never overwrite, always add a version



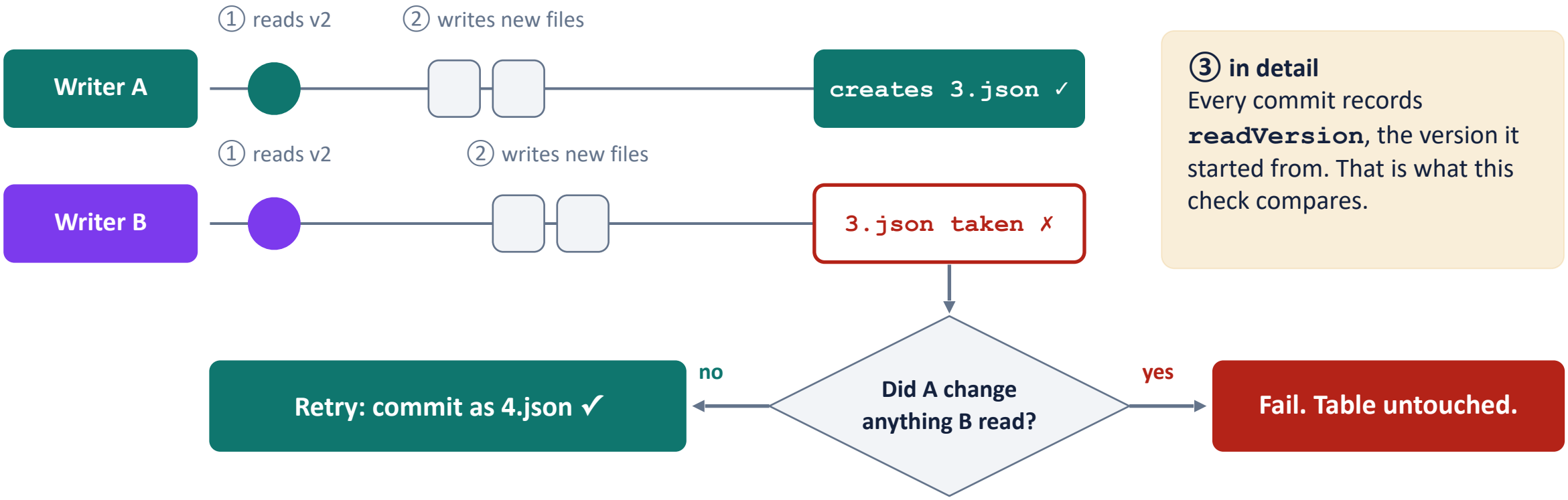
Note the version and the files you read

part-A untouched. part-B invisible until ③.

Still on disk. v1 still readable.

Every operation	Writes a new data file…	…and logs
INSERT	with the new rows	`add new`
UPDATE	copy of the old file, rows changed	`remove old, add new`
DELETE	copy of the old file, rows left out	`remove old, add new`
Compaction	one big file from many small ones	`remove many, add one`

Two writers at once: optimistic concurrency

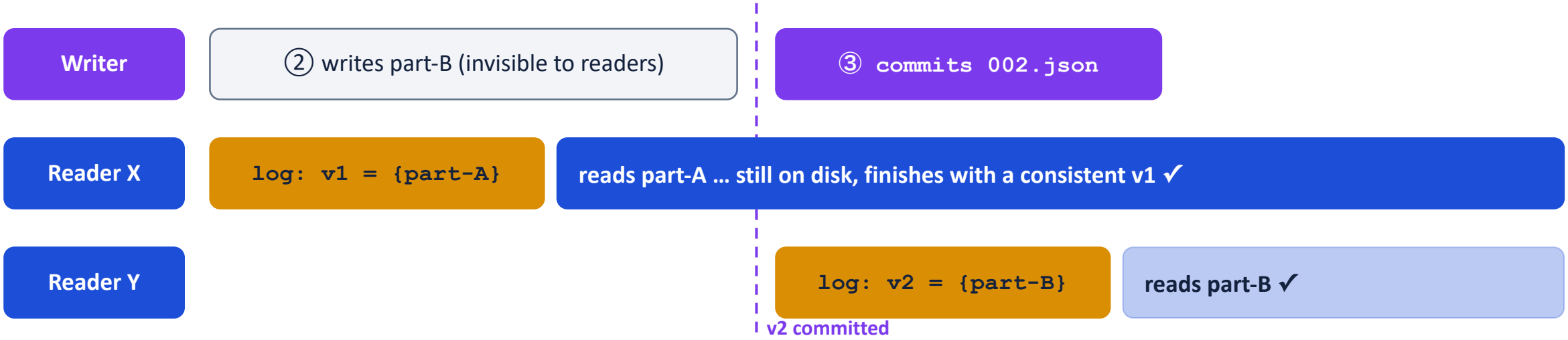


③ in detail
Every commit records **`readVersion`**, the version it started from. That is what this check compares.

No locks. Bet that writers touch different data, and check before committing.

Switch to notebook 02a

Readers never wait: each reads its own version's files



Without MVCC the writer overwrites part-A in place. Reader X, halfway through, gets the first rows old and the rest new: a table that never existed.

```
1 Sports    (old)
2 News      (old)
3 ???       (new)
```

Old files kept → old versions readable → time travel, for free

A partner sends a wrong age

id	gender	age	interest
1	Male	25	News
2	Female	999	News
3	Male	35	Social
4	Female	28	Fashion

999 is a perfectly valid integer. Nothing stops it.

1. **Can we see the table as it was before?**
2. **Can we undo it?**
3. **Can we stop it happening again?**

Switch to notebook 02b

ACID: what the log and versions gave us

A

All of a commit, or none of it

The transaction log controls atomicity: one commit is one N.json file.

Seen: Update = remove + add in one file (01). The loser's Parquet file: NEVER COMMITTED (02a).

C

Only ever from one valid version to the next

Valid = schema + rules you declare. Conflicting writes are rejected.

Seen: CHECK rejected age 999, no new version (02b). The losing update failed instead of corrupting (02a).

I

Readers: snapshot isolation. Writers: serializable.

Each reader reads its own version's files. Writers ordered by readVersion.

Seen: Reader X kept reading v1 (MVCC slides). readVersion in the history (02a).

D

Committed means permanent

Commit = creating N.json (only one writer can) on durable cloud storage.

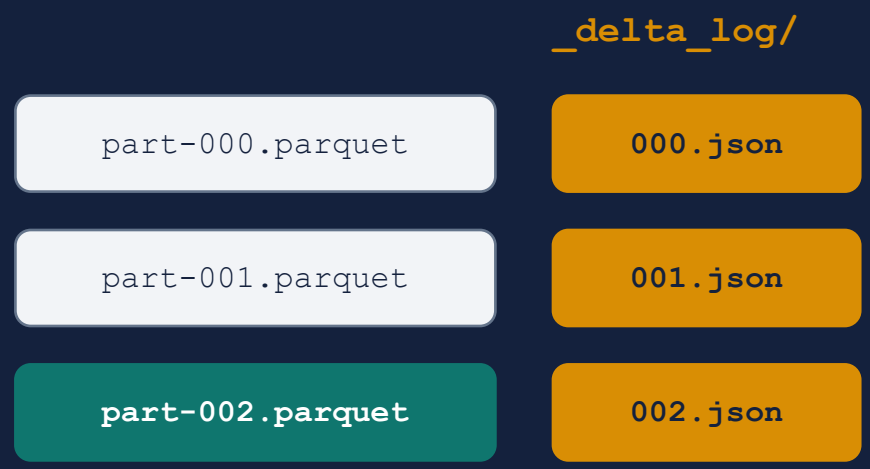
Seen: The winner's commit stayed. The loser could not overwrite it.

And because old files stay: `VERSION AS OF 1` to look back, `restoreToVersion(1)` to undo, without copying data.

Next: real tables have thousands of files. Do we have to read all of them?

Skipping and MERGE

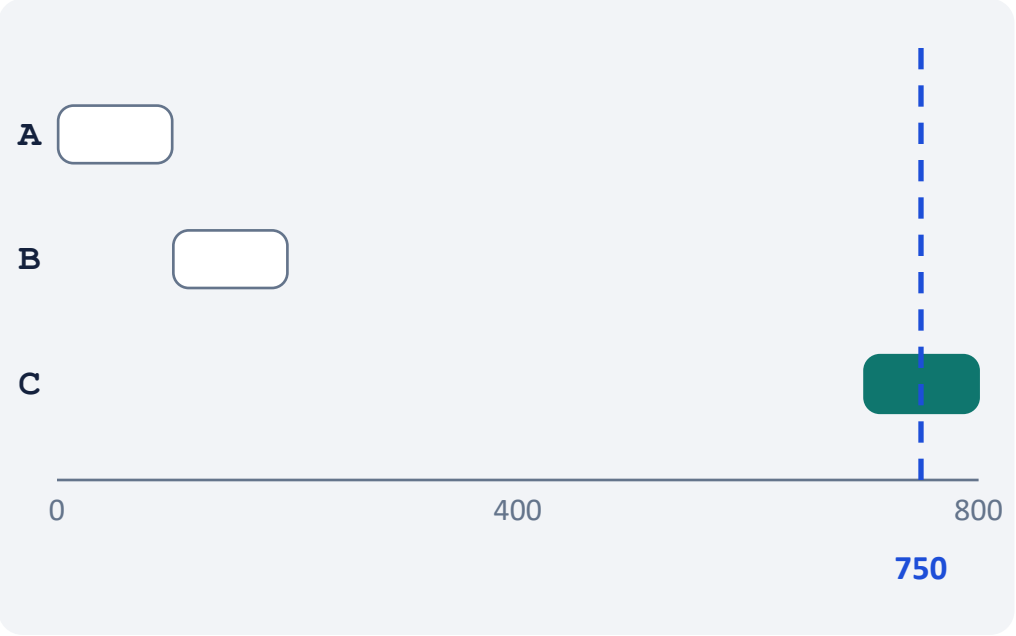
Read only the files that matter. Rewrite only the files that matter.



Min and max let us rule files out

```
WHERE id = 750
```

File	min id	max id	Could it contain 750?
A	1	100	✗ skip
B	101	200	✗ skip
C	701	800	✓ maybe: read it



The stats live in the log, so files are ruled out before any of them is opened.

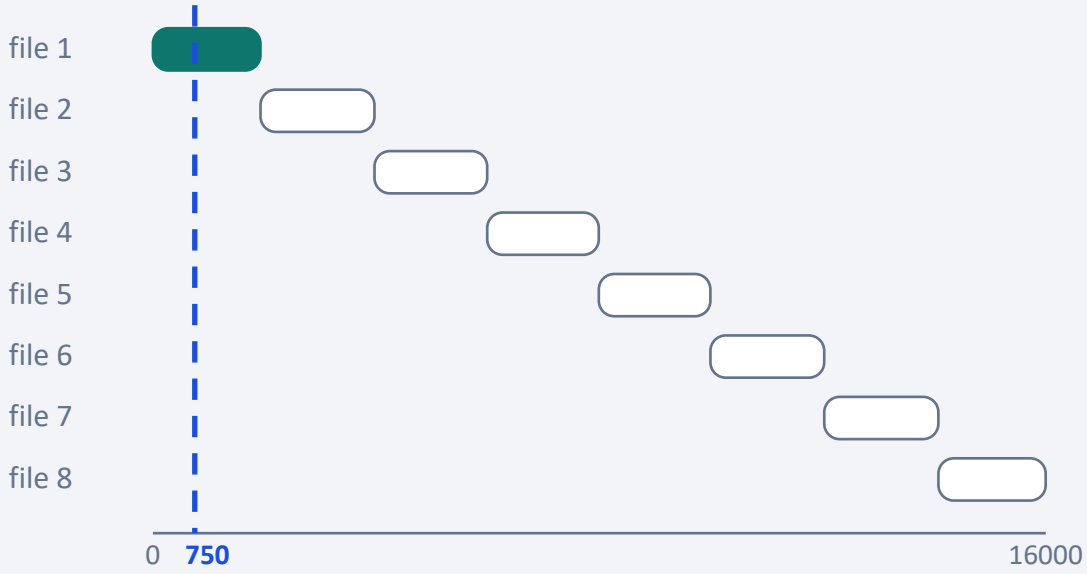
Same data, different layout

SPREAD: rows scattered randomly



Every range contains 750 → read 8

ORDERED: each file holds a range of ids



Only one range contains 750 → read 1

Stats only help when the layout gives each file a narrow range.

Switch to notebook 03 · Part 1

A partner sends a batch

Our table: three files

A	100, **150**, 200
B	75, 125, 250, 300
C	400, 450, 500

The batch

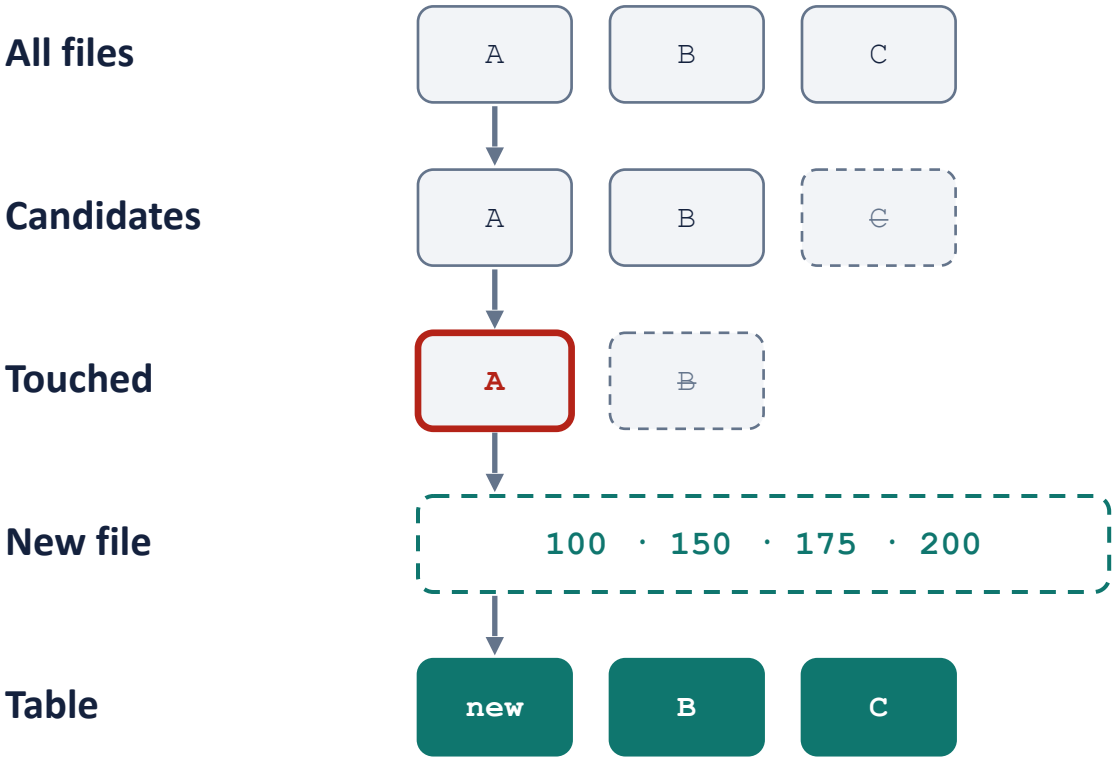
id	interest	
150	Fashion	already exists → **update**
175	Social	new → **insert**

Three cases every upsert has to handle

In table?	In batch?	Action
✓	✓	**update**
✗	✓	**insert**
✓	✗	**keep**

One full outer join gives all three cases.
But writing its result rewrites all 11 rows in all three files, to change one user and add one.

MERGE in two phases



1. **Skip with stats:** ids 150–175 can't be in C. C is never opened.
2. **Inner join with the batch:** only A has a match. B was read, not rewritten.
3. **Full outer join of A's rows:** 100 keep · 150 update · 175 insert · 200 keep

`commit: remove A, add new` *B and C never rewritten*

A file can be worth reading without being worth rewriting.

Switch to notebook 03 · Part 2

What we saw

	What it does	How you get it
Data skipping	Rules files out using min/max in the log	Automatic, on every read and in MERGE's first phase
Clustering	Puts related values in the same files, so ranges are narrow	`OPTIMIZE … ZORDER BY (col)`, or liquid clustering: `CLUSTER BY`
Compaction	Merges many small files into fewer big ones	`OPTIMIZE`
MERGE	Skip → find touched files → rewrite only those	One command, one commit

Next: the dashboard downstream needs to know what changed.

Three questions, three reads

Question	Read
What is true now?	current table
What was true at version 3?	time travel `versionAsOf`
What changed between versions?	**Change Data Feed**

User 1 moves Sports → News



The dashboard needs the row before and after.

Time travel shows what the table was. The change feed shows what changed.

Switch to notebook 04

Changes don't last forever



✗ log entries expire after 30 days

✗ VACUUM deletes old files after 7 days

The change feed has to be **switched on before** the changes happen.
It reads from the log and old files, so it only reaches back as far as they are kept.

Back to where we started

The problem	What fixed it
Half a write	Nothing is in the table until its commit exists (Module 1)
A file disappears	The log says which files the table needs; readers fail loudly instead of silently returning less (Module 1)
Change one row	Write a new file, swap it in one commit (Modules 1–3)

A Delta table is Parquet files plus a log that says which files are the table right now.

Blogs Change Data Feed without the Delta log · CDF failure scenarios, prevention and recovery · MERGE optimisation strategies